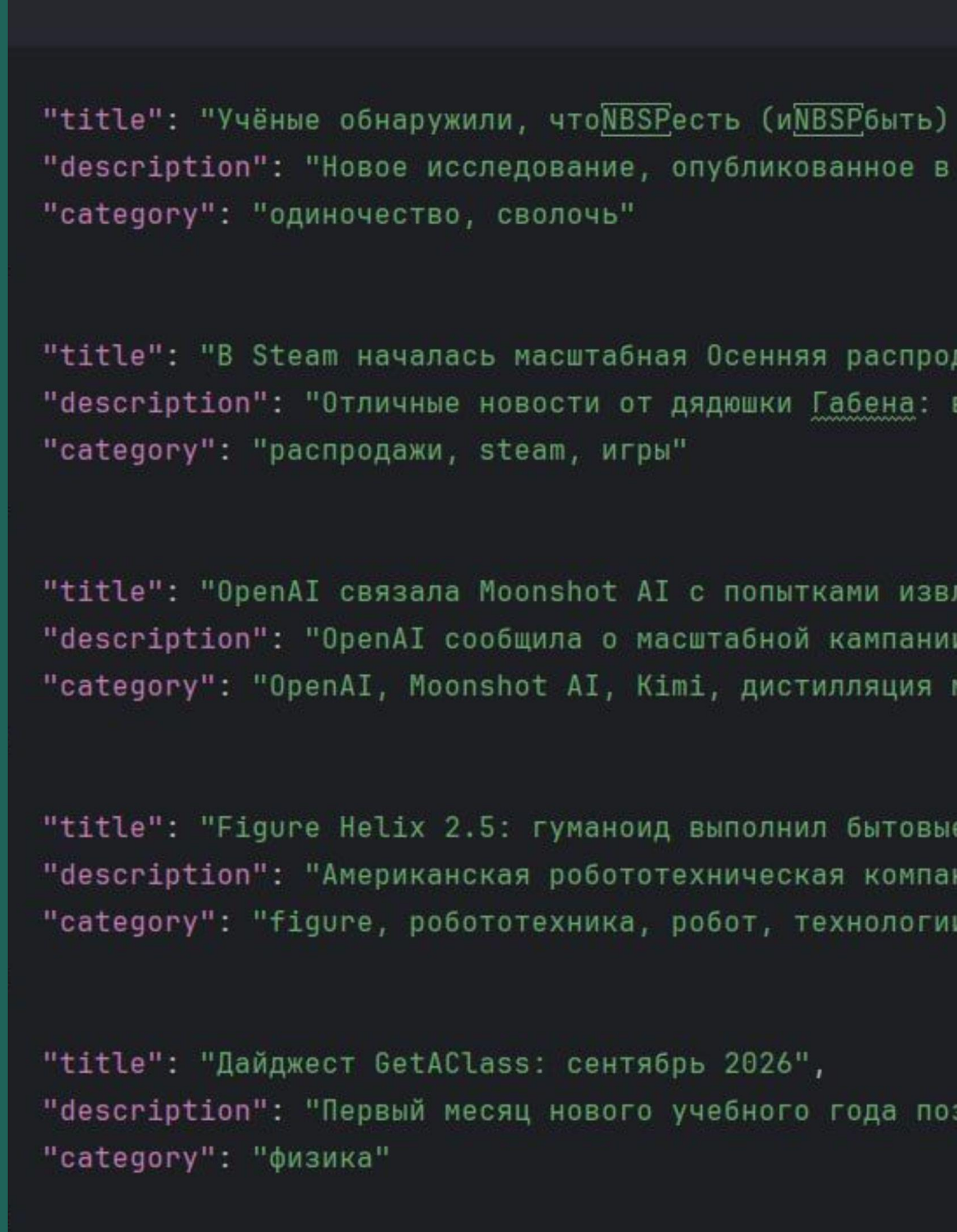
AI NEWS ARCHITECTURE · 2026

ИИ-ассистент для персонализированного новостного дайджеста

Система собирает новости, понимает интересы пользователя и превращает поток RSS в короткий готовый дайджест.

RSS-поток — Очистка — Векторы — Дайджест

Python · feedparser · sentence-transformers · cosine similarity · Markdown / DOCX



КОНТЕКСТ И ПРОБЛЕМА

Новости уже не помещаются в человеческий день

Источников много, а времени на ручный отбор мало.

Ключевые слова пропускают новости, где смысл есть, а нужного слова нет.

Нужен конвейер: сбор → очистка → смысловой отбор → дайджест.

СМЫСЛОВОЙ ПОИСК VS ПОДСТРОКА

Главная идея: искать смысл, а не совпадение слов

Интерес «Информатика и программирование» должен находить новости про Python и ML, даже если само слово «информатика» не встречается в тексте.

ВХОДНОЙ СИГНАЛ

Вход

Интересы пользователя + поток RSS:
неструктурированные тексты, темы и векторные описания приоритетов.

Сырые статьи | Вектор тем

ВЫХОДНОЙ РЕЗУЛЬТАТ

Топ-5

5 наиболее релевантных новостей:
отранжированный список с высокими метриками семантической близости.

Точный дайджест | score ≥ 0.45

АРХИТЕКТУРА СИСТЕМЫ

Пайплайн собирает весь путь в одну систему

Последовательная обработка превращает сырые веб-ленты в готовый структурированный отчет.

01

RSS-потоки

Habr и внешние новостные ленты.

02

Парсинг

Сбор через feedparser и requests.

03

Очистка

Удаление HTML и нормализация.

04

Теги

Категории и метаданные статей.

05

Фильтрация

Embeddings и cosine similarity.

06

Выгрузка

Генерация Markdown / DOCX

Конвейер исключает человеческий фактор: каждый этап валидирует данные перед передачей следующему модулю.

СТРУКТУРА ДАННЫХ

Единый JSON связывает парсер, ИИ и сайт

Каждая новость получает одинаковый контракт: `source`, `title`, `description`, `category`, `date` и `link`. Благодаря этому модули можно менять независимо.

- `source`
- `title`
- `description`
- `category`
- `date`
- `link`


```
[
    {
        "title": "Учёные обнаружили, чтоNBSPесть (иNBSPбыть) вNBSPо
        "description": "Новое исследование, опубликованное в Scienc
        "category": "одиночество, сволочь"
    },
    {
        "title": "В Steam началась масштабная Осенняя распродажа",
        "description": "Отличные новости от дядюшки Габена: в Steam
        "category": "распродажи, steam, игры"
    },
    {
        "title": "OpenAI связала Moonshot AI с попытками извлечь ск
        "description": "OpenAI сообщила о масштабной кампании по из
        "category": "OpenAI, Moonshot AI, Kimi, дистилляция моделей
    },
    {
        "title": "Figure Helix 2.5: гуманоид выполнил бытовые задачи
        "description": "Американская робототехническая компания Fig
        "category": "figure, робототехника, робот, технологии, буду
    },
    {
        "title": "Дайджест GetAClass: сентябрь 2026",
        "description": "Первый месяц нового учебного года позади, в
        "category": "физика"
    },
```

```python
def CleanHtml(raw_html: str) -> str:  1 usage
    if not raw_html:
        return ""
    CleanText = re.sub( pattern: r'<[^>]+>',  repl: '', raw_html)
    CleanText = unescape(CleanText)
    return ' '.join(CleanText.split())

def ParseAndSave(output_file: str = "news.json"):  1 usage
    all_news = []

    for source_name, rss_url in SOURCES.items():
        feed = feedparser.parse(rss_url)

        for entry in feed.entries:
            title = entry.get("title", "").strip()
            raw_description = entry.get("summary") or entry.get("description", "")
            description = CleanHtml(raw_description)
            tags = []
            if "tags" in entry:
                tags = [tag.term for tag in entry.tags if hasattr(tag, "term")]
            category_str = ", ".join(tags) if tags else ""
            news_item = {
                "title": title,
                "description": description,
                "category": category_str
            }
            all_news.append(news_item)

    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(all_news, f, ensure_ascii=False, indent=4)
```

ПРЕДОБРАБОТКА ТЕКСТА

Очистка превращает HTML в текст для ИИ

1. feedparser читает RSS.
2. Регулярное выражение удаляет HTML-теги.
3. `html.unescape` превращает сущности вроде ` ` в нормальный текст.
4. Результат сохраняется в JSON.

КОНТЕКСТ НОВОСТИ

Теги добавляют новости контекст

- Теги извлекаются из HTML и со страницы источника.
- `set()` убирает дубликаты.
- Категория становится дополнительным сигналом для отбора.

```
ExtractTagsFromHtml(raw_html: str) -> list:  2 usages
if not raw_html:
    return []

pattern = r'<a\s+[^>]*href=["\'][^"\']*["\'][^>]*>\s*([^<\s]+)'
matches = re.findall(pattern, raw_html)

return list(set(matches))


GetVcTagsFromPage(url: str) -> list:  1 usage
if not url:
    return []

try:
    resp = requests.get(url, headers=HEADERS, timeout=15)
    resp.raise_for_status()

    return ExtractTagsFromHtml(resp.text)

except requests.exceptions.RequestException:
    return []


ParseAndSave(output_file: str = "news.json"):

all_news = []

for source_name, rss_url in SOURCES.items():
    try:
        feed = feedparser.parse(rss_url)
```

СЕМАНТИЧЕСКИЙ СКОРИНГ

Эмбеддинги оценивают релевантность новости

sentence-transformers

Переводит интерес и новость в векторы.

cosine_similarity

Сравнивает их смысловую близость.

Порог score ≥ 0.45

Пропускает подходящие материалы.

Ограничение Топ-5

Цикл останавливается после первых 5 релевантных новостей.

score ≥ 0.45 **Топ-5 в дайджест**

```python
def calculate_relevance(news_text, user_preferences):  1 usage
    texts = [user_preferences, news_text]
    embeddings = model.encode(texts, normalize_embeddings=True)

    score = cosine_similarity( X: [embeddings[0]], Y: [embeddings[1]])[0][0]

    return float(score)


def main():  1 usage
    user_tags = "Информатика и программирование"
    data = load_news(FILE_NAME)
    print('START')

    while data:
        list_news = []

        # Ищем первую подходящую новость
        for article in data:
            title = article.get("title", "")
            tags = article.get("category", "")
            date = article.get("date", "")
            link = article.get("link", "")

            score = calculate_relevance(tags, user_tags)
            if score >= 0.45:
                list_news.append(title)

            if len(list_news) == 5:
                break
```

Дайджест новостей

Тема: Информатика и программирование

1. Selectel выпустил новую версию aish — ИИ-агента для работы в терминале

- - Дата: 01.10.2026 09:07
- - Ссылка: https://habr.com/ru/companies/selectel/news/1087590/?utm_source=habrahabr&utm_medium=rss&utm_campaign=1087590
- - Приоритет: Высокий

2. UserGate назначил вице-президентом по HR экс-топ-менеджера Cisco Елену Щедрову

- - Дата: 01.10.2026 08:48
- - Ссылка: https://habr.com/ru/companies/usergate/news/1088972/?utm_source=habrahabr&utm_medium=rss&utm_campaign=1088972
- - Приоритет: Высокий

3. Вышел Tails 7.14

- - Дата: 01.10.2026 08:43
- - Ссылка: https://habr.com/ru/news/1088568/?utm_source=habrahabr&utm_medium=rss&utm_campaign=1088568
- - Приоритет: Высокий

ИТОГОВЫЙ ПРОДУКТ

Прототип уже формирует читаемый дайджест

На выходе пользователь получает подборку новостей с датами, ссылками и приоритетом. Формат легко открывается как Markdown или DOCX.

5

Отобранных новостей в каждом выпуске без информационного шума

2

Формата выгрузки: DOCX для документов

Готово к внедрению в рабочий процесс читателя

ВАЛИДАЦИЯ И QA

Проверяем не только код, но и пользовательский путь

Критерии готовности охватывают стабильность сбора, качество очистки и удобство чтения.

2+ источника

Парсер стабильно собирает валидный JSON из разных каналов.

Чистый текст

В описаниях отсутствуют HTML-теги и сущности .

Смысловой фильтр

Релевантные статьи проходят отбор, посторонние отсекаются.

Интерфейс

Документ показывает 5 новостей и фильтрует их по категориям.

Доступность

Ссылки кликабельны, файлы Markdown и DOCX открываются.

Система проверена end-to-end: от сетевого запроса к ленте до открытия файла дайджеста пользователем.